Strings, Parsing and Recursion

Topic 6 of 10

20 questions | skeleton and writing space only

Each question: the prompt, the function or class signature, and ruled lines to write your own solution.

1. Find the Longest Straight Run of One Color

HARD | asked 2026-08-23 |

Given a rectangular grid of colors, return the length of the longest consecutive sequence of cells with the same color along a horizontal, vertical, or diagonal straight line. A sequence cannot change direction or skip a cell. Both diagonal orientations count. The length is the number of cells in the sequence. ### Input - grid: an array of strings of equal length. Each character represents one cell's color. ### Output Return the maximum length as an integer.

```
def longest_run(grid):
```

2. Matrix Flower Placement with House Adjacency

HARD | asked 2026-05-31 | grid, backtracking

Examples: H 0 0 0 H 0 0 0 0 0 0 H 0 0 0 0 The task is to fill some 0 cells with F while satisfying both the row/column constraint and each house's adjacency requirement.

```
def place_flowers(grid, exactly_one=False, find
  _all=False):
```

3. Convert Valid Snake Case to Lower Camel Case

HARD | asked 2026-04-16 |

 # Convert Valid Snake Case to Lower Camel Case Given a valid snake-case identifier, convert it to lower camel case. The input consists of one or more lowercase ASCII words separated by single underscores. There is no leading underscore, trailing underscore, or pair of adjacent underscores. The output keeps the first word lowercase, removes each underscore, and capitalizes the first letter of every later word. Implement: python def solve(name: str) -> str: ...

```
def snake_to_camel(s):
```

4. Find Dictionary Matches for a Jammed String

HARD | asked 2026-04-09 |

Overview: This question evaluates string-processing and pattern-matching skills, specifically run-length grouping of consecutive characters and efficient dictionary lookup, within the Coding & Algorithms domain.

```
class JammedIndex:

    def __init__(self, words):
```

```
    def matches(self, query, min_stretch=3):
```

```
    def fits(q, w, min_stretch):
```

5. Mahjong Winning Hand Detection (14 Tiles)

MEDIUM | asked 2026-10-09 | recursion, simulation

Mountain View onsite coding round 1: given a hand of 14 tiles, decide whether it is a winning mahjong hand. Input: array of 14 integers representing tile values (clarify suit / range during the interview). A winning hand consists of: One pair (two identical tiles), and Four melds, each being either: A triplet (3 identical tiles), or A run (3 consecutive ranks in the same suit).

```
def is_winning_hand(hand, melds=4):
```

6. Count Prefix-Matching Simple Paths in a Labeled Graph

MEDIUM | asked 2026-08-09 |

Overview: Count distinct simple graph paths whose labels match every nonempty prefix of a query word. Paths may start anywhere but cannot revisit a vertex, so the task rewards subset-state search and careful counting over a small labeled graph.

```
def prefix_paths(labels, edges, query, undirect
  ed=False):
```

7. Snake Case to Camel Case with Repository-Wide Refactor

MEDIUM | asked 2026-07-14 | string, refactoring

Convert a valid snake_case identifier to camelCase, then design a safe repository-wide migration for variables and function names without changing third-party symbols, public interfaces, or the tests' ability to detect regressions. Implement a conversion from a valid snake_case identifier to camelCase. The base input is guaranteed valid: no leading underscore, trailing underscore, or doubled separator. Example: foo_bar -> fooBar.

```
def snake_to_camel(name):
```

8. Check Whether Digits Are Monotone Increasing

MEDIUM | asked 2026-07-04 |

Overview: Check whether a non-negative integer's decimal digits are non-decreasing from left to right.

```
def is_monotone_digits(n):
```

9. Can board states be transformed?

MEDIUM | asked 2026-04-01 |

Overview: This question evaluates reasoning about constrained piece movement, string transformation, and invariant-based correctness in the Coding & Algorithms domain, testing skills in string manipulation and combinatorial reasoning while examining both conceptual properties and practical algorithmic implementation.

```
def can_transform(start, end):
```

10. Google Doc Line Wrap (Word Wrap with Whitespace and Newlines)

MEDIUM | asked 2026-02-01 | string, simulation

Examples: text = "Hello world\nFoo bar baz qux" width = 10 Line 1: "Hello" (5 chars; "world" + space would be 11 -> break) Line 2: "world" Line 3: "Foo bar" (7 chars; "baz" -> "Foo bar baz" = 11 -> break) Line 4: "baz qux" Output: 4

```
def wrap_count(text, width):
```

11. Decide string transform with directional tokens and walls

MEDIUM | asked 2025-07-15 |

You are given two equal-length strings start and target of length n, each consisting only of the characters 'L', 'R', '.', and 'X'. A dot '.' represents an empty slot; 'X' represents an immovable wall; 'L' and 'R' are tokens that may move across empty slots subject to these rules: (1) An 'L' token may move any number of positions left into '.', but may never move right, pass through another token, or pass through an 'X'.

```
def can_transform(start, end):
```

12. Permutations

MEDIUM | asked n/a |

Given an array of distinct integers, return all possible permutations of the array. A permutation of an array is a rearrangement of its elements. Example Input: nums = [1, 2, 3] Output: [[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]

```
def permute(nums):
```

13. Generate Parentheses

MEDIUM | asked n/a |

Generate all combinations of well-formed parentheses. A parenthesis combination is well-formed if each opening parenthesis "(" is closed by a matching closing parenthesis ")" in the correct order. For example, "()", "()()", and "((()))" are all combinations of well-formed parentheses, while ")(", "((" and "(()" are not.

```
def generate(n):
```

14. Given an array of strings, return groups of "buddy strings," where the strings have the same length and the distance between corresponding characters is the same.

MEDIUM | asked n/a |

Given an array of strings, return groups of "buddy strings," where the strings have the same length and the distance between corresponding characters is the same.

```
def group_buddies(words, key=shift_key):
```

15. Write functions to serialize and deserialize a list of strings.

MEDIUM | asked n/a |

Serialize and Deserialize a List of Strings. You are tasked with writing two functions: serialize and deserialize. The serialize function will take a list of strings as input and return a single string. The deserialize function will take this single string and reconstruct the original list of strings. Details The serialize function should concatenate all strings in the list into a single string, with each original string's length encoded as a single character prefix.

```
def serialize(items):
```

16. Explain how to find a target sum in an array.

MEDIUM | asked n/a |

Determine the Number of Ways to Assign Symbols to Reach a Target Sum. You are given a list of non-negative integers, a1, a2, ..., an, and a target sum S. Your task is to assign each integer in the list a symbol + or - and calculate how many different ways there are to arrange these symbols such that the sum of the integers equals S after applying the assigned symbols.

```
def count_signs(nums, target):
```

17. Determine if two sentences are similar.

MEDIUM | asked n/a |

Determine if Two Sentences Are Similar. Two sentences are similar if they have the same length and each pair of corresponding words in the two sentences is similar. The similarity between words is defined by the provided list of similar word pairs. A word is always similar to itself.

```
def similar_direct(s1, s2, pairs):
```

18. Implement string formatting and request tracking tasks

EASY | asked 2026-02-11 |

Overview: This multi-part question evaluates string manipulation, numeric formatting, character-mapping validation under 180deg rotation, and event-tracking/order management competencies, assessing parsing, edge-case handling, mapping validation, and design of data structures for ordered output.

```
def snake_to_camel(s):
```

19. Given an array of strings, return the words that can be typed using letters of the alphabet on only one row of American keyboard.

EASY | asked n/a |

Given an array of strings, return the words that can be typed using letters of the alphabet on only one row of American keyboard.

```
def one_row_words(words, rows=QWERTY):
```

20. Return all unique substrings of a given string.

EASY | asked n/a |

Return all unique substrings of a given string.

```
def unique_substrings(s):
```